

Adaptive Continuous Authentication

Account-Takeover Detection Using Behavioral Intelligence

Anjali Kumari (1024030457)
Yatharth Kansal (1024030456)
Ridhi Batra (1024030464)

Project Proposal for UCS503P
Submitted to: Prof. Asif
Thapar Institute of Engineering and Technology
12 September 2026

Project Overview

Adaptive Continuous Authentication aims to detect changes in the person using an already-authenticated session through typing behavior. The current course prototype connects keyboard timing capture, feature extraction, an Isolation Forest model, a trust scorer, and a Flask dashboard. The wider signal-fusion and enforcement capabilities below are proposed extensions.

1 Higher-order goal

This project strengthens session-level trust in authenticated systems by continuously verifying that the person behind the keyboard is still the legitimate user, not just at login. The engineering objective is to translate **behavioral risk signals** into a measurable, deployable trust-scoring service.

2 Time-to-value

- Deliver a working end-to-end pipeline early — capture, baseline, live score — before perfecting any one detection model.
- Iterate weekly, starting with a single behavioral signal (keystroke dynamics) before fusing in more.
- Success in the first iteration is a live 0–100 trust score for one signal; later iterations add signals and refine accuracy.

3 Problem Statement

Traditional authentication is a one-time gate: credentials and MFA are checked once at login, after which the session is marked “trusted” and left unmonitored. This gap lets attackers exploit:

- **Session hijacking:** a stolen token inherits full trust without presenting credentials.
- **Unattended workstations:** a logged-in session can be used by anyone physically present.

- **Binary trust:** sessions are simply “trusted” or not, with no way to react as risk builds mid-session.

Privilege escalation and data exfiltration during an already-authenticated session routinely bypass login-time defenses, since nothing re-checks identity once the door is open.

4 Proposed Solution

4.1 Overview

Build a **continuous, adaptive trust engine** that fuses behavioral, system, and contextual signals into a single live session-trust score (0–100), and reacts automatically as it changes: **80–100** continue normally, **60–79** monitor closely, **40–59** step-up MFA, **<40** restrict/alert. Components: client-side signal capture (keystroke, mouse, app usage), a baseline & fusion engine, anomaly/sequence detection, and the trust-scoring service driving these responses.

4.2 Core workflow

1. User authenticates normally; the session begins with an initial trust score.
2. A client agent streams behavioral/system events to the backend.
3. The fusion engine compares events against the user’s baseline and updates the score in real time.
4. The system acts per the current score band (monitor, step-up MFA, or restrict/alert).

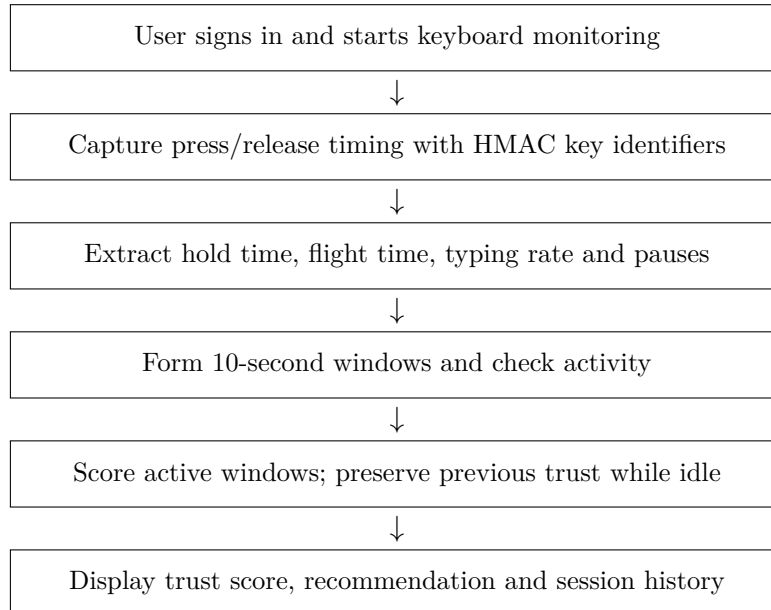


Figure 1: End-to-end workflow of the current keyboard prototype.

4.3 Current prototype and response policy

The implemented prototype uses keyboard input only. Browser monitoring is limited to the dashboard typing area; a separate desktop collector supports local recording. The model currently uses six active recorded windows, so its output is a demonstration of the pipeline, not measured identity accuracy. Login does not enroll a personal baseline.

The current implementation uses three score bands: trust at least 70 allows the session, trust from 40 to below 70 recommends re-authentication, and trust below 40 recommends locking. These replace the earlier four-band design for the present demonstration. The dashboard reports recommendations; actual MFA and session-lock enforcement remain future work. Inactive windows preserve the previous score, with no score shown before the first active observation.

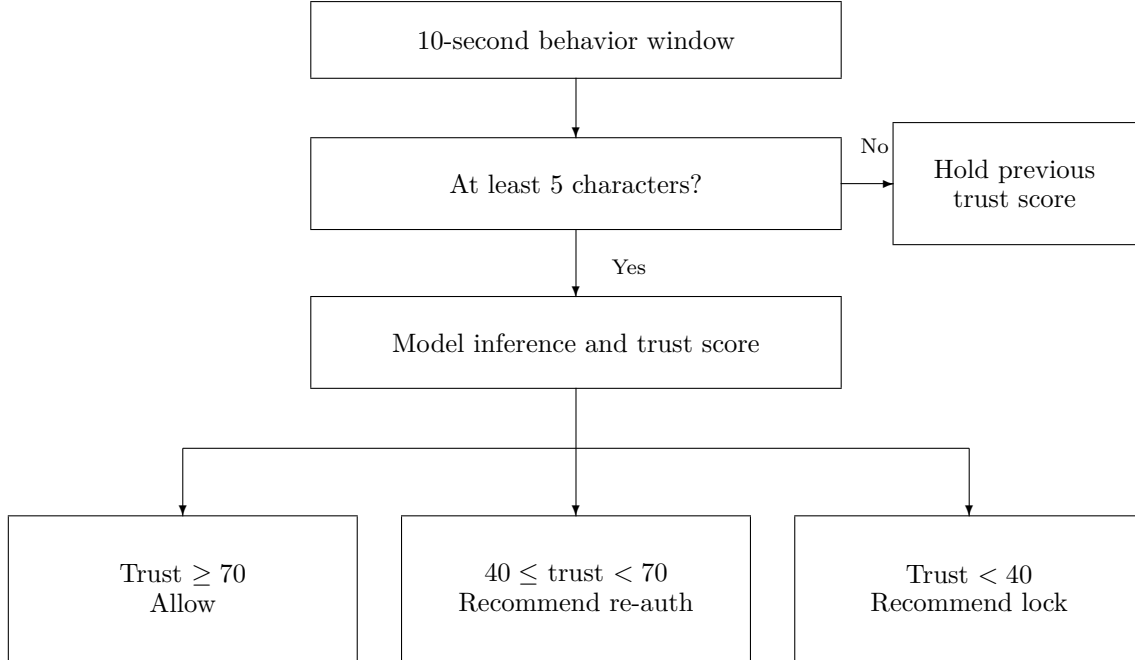


Figure 2: Activity handling and the implemented authentication decision flow.

5 Solution Approach

This is an engineering/applied-ML course project, so we focus on a working, evaluable pipeline rather than novel research in behavioral biometrics.

- **Signal modeling:** start with tractable features (keystroke dwell/flight time, mouse velocity); use lightweight statistical or one-class-classifier baselines before heavier models. Anomaly scores feed the response layer — never auto-terminate on one weak signal.
- **Architecture:** a client agent (capture), a scoring backend API (baselines, scoring, responses), and a database (baselines, event history, score timelines).
- **CI/CD:** automated build/test on every change, repeatable staging deploys, enabling safe frequent releases as signals are added.

6 Evaluation Criterion

6.1 Primary metric

Detection latency: time between an injected anomalous behavior (simulated takeover) and the score crossing into the step-up/restrict band. Target: a small, bounded number of events during pilot testing, measured from event vs. score-change timestamps.

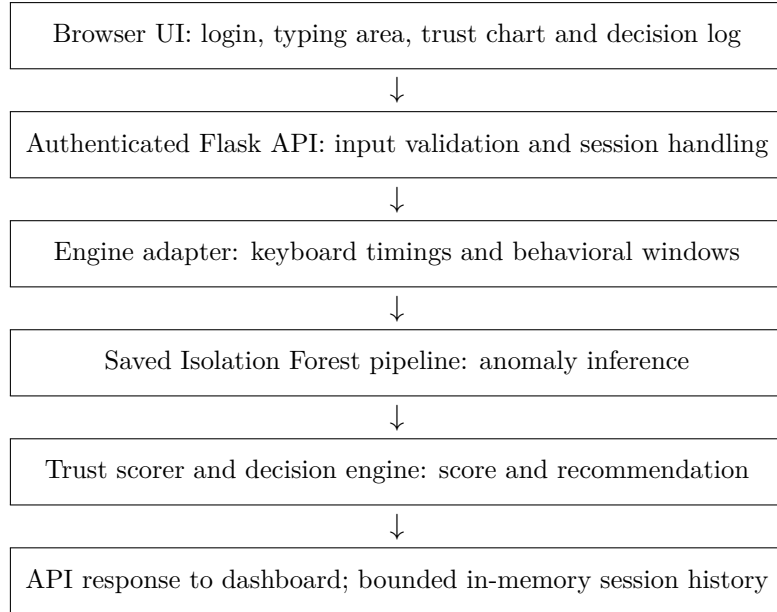


Figure 3: Current system architecture. A database and additional signals are future extensions.

6.2 Secondary metrics

- **False-positive / false-negative rate** of the score bands.
- **Baseline convergence time** for a new user.
- **Reliability** of the scoring service during pilot/demo sessions.

6.3 Pilot validation plan

Recruit a small group of pilot users to build genuine baselines, then simulate takeovers (a second person continuing a logged-in session) to test detection latency and false-negative rate across iterations.

7 Scalability

- **Theoretically:** decouple signal capture from scoring, process events via a queue/stream, and keep the scoring API stateless where feasible.
- **Practically:** run on common infrastructure — managed database, containerized deployment, CI/CD for staging/production.

8 Engine availability heuristic

Mature tooling covers this project’s needs: a web/backend framework for the API and dashboard, a managed database for event/session storage, classical ML libraries for baselines and anomaly detection, browser/desktop APIs for keystroke and mouse capture, a test framework, and a CI runner with staging target. We use these to focus on signal design, fusion logic, and evaluation.

9 Project scope and deliverables

9.1 Initial deliverable

- Client-side capture of keystroke dynamics; baseline for a single test user
- Real-time trust-score computation (0–100) with a minimal live dashboard
- CI: build + backend unit tests + linting; CD to staging

9.2 Subsequent deliverables

- Mouse dynamics and app/process usage signals, fused into one score
- Adaptive response tiers wired to real actions (step-up MFA, session restriction)
- Anomaly vs. attack-chain (sequence-level) detection
- Pilot evaluation dashboard for detection latency and error rates

10 Risks and mitigations

- **Privacy/consent:** collect only what is needed, anonymize where possible, get informed pilot-user consent.
- **Baseline cold-start:** use a conservative initial score and a defined learning period for new users.
- **False positives frustrating users:** keep step-up MFA lightweight; tune thresholds from pilot feedback.
- **Measurement ambiguity:** define score bands and log all timestamps needed for detection-latency measurement before development begins.

11 Summary

This project proposes a continuous, adaptive authentication system that scores session trust in real time from behavioral, system, and contextual signals, reacting through tiers from passive monitoring to session restriction. It emphasizes time-to-value through signal-by-signal iteration, CI/CD for safe frequent releases, and measurable evaluation (detection latency, false-positive/negative rates) — a working, evaluable pipeline rather than research-grade novelty in behavioral biometrics.